



Aqsa Khalid
Lecturer
School of Computing

CS3002 Information Security

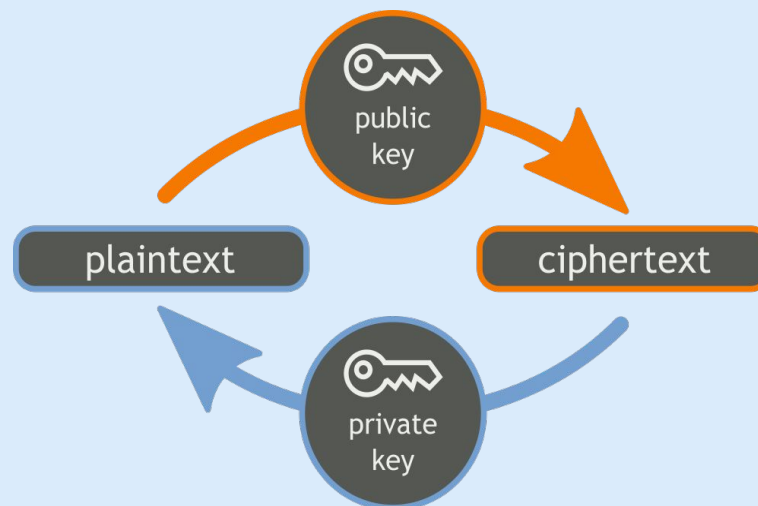


Public Key Cryptography

Asymmetric Cryptography



- Newer form of crypto, invented in 1970s
- Uses a pair of keys for each user: one public, one private
- Also known as Public Key Cryptography (PKC)
- The private key can unlock (decrypt) what is locked (encrypted) with the public key and vice versa

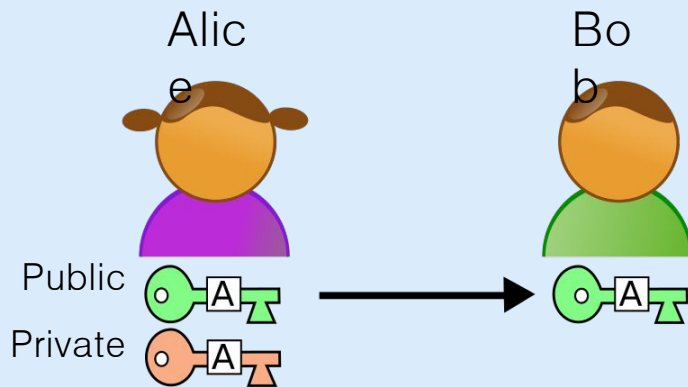


Asymmetric Cryptography

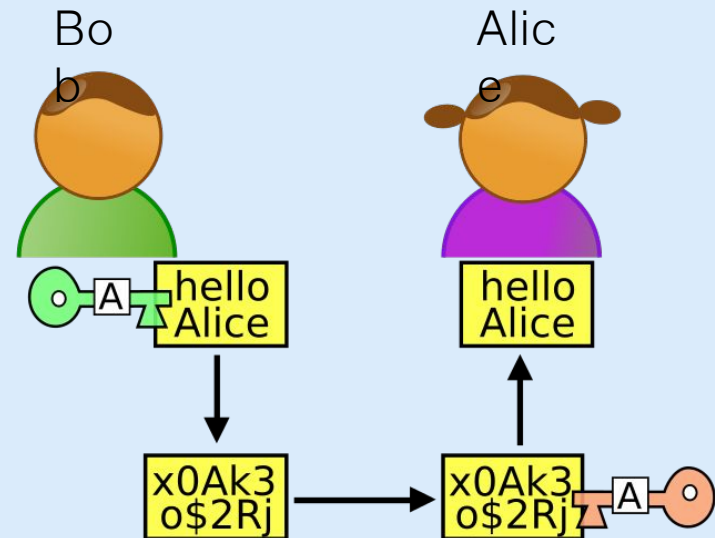


Use Case 1: Data encryption

Aiming for confidentiality



Step1: Announce the public key



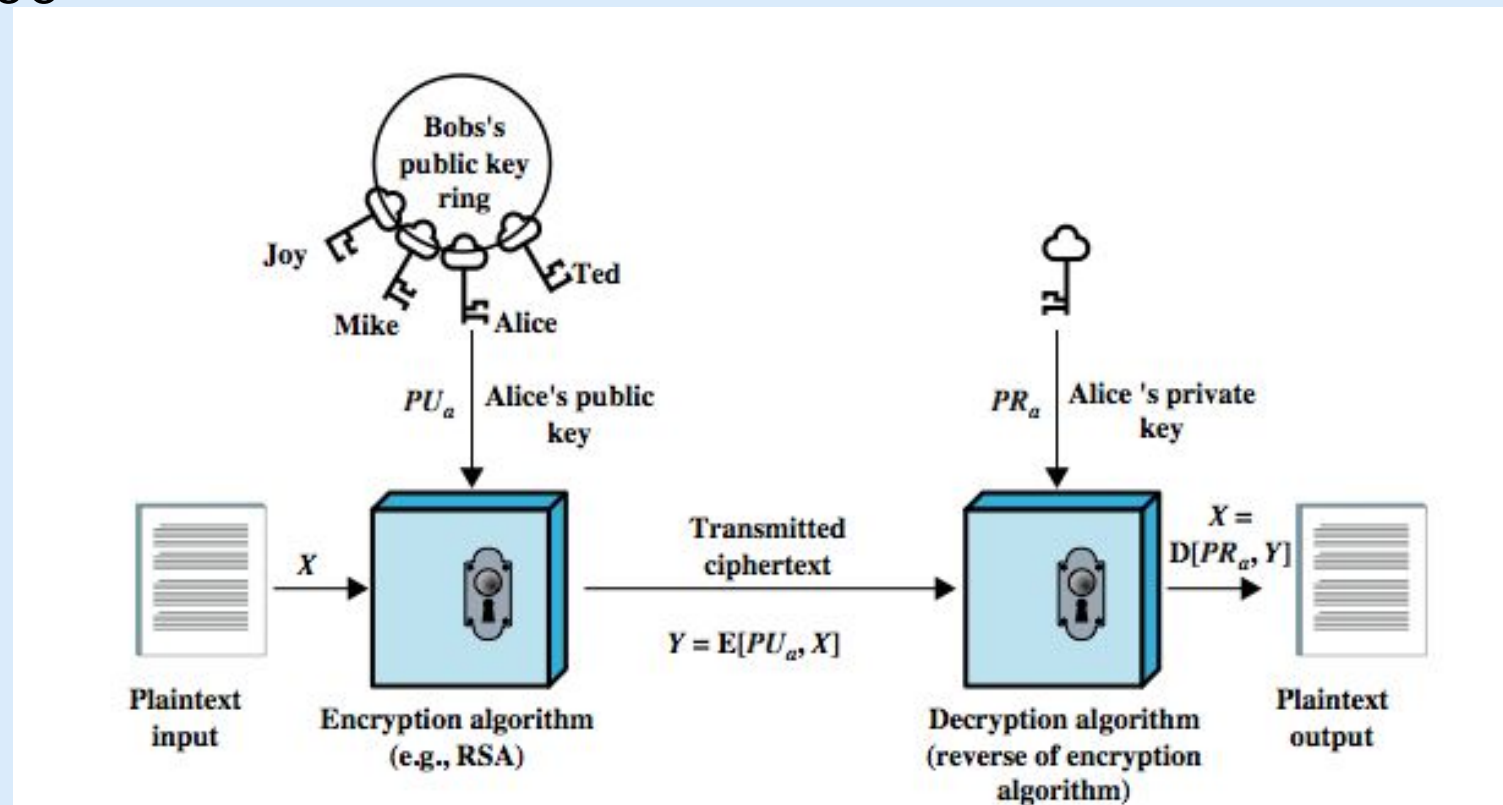
Step2: Encrypt with recipient's public key

Step3: Decrypt with private key

Asymmetric Cryptography



Workflow of Bob sending a confidential message to Alice



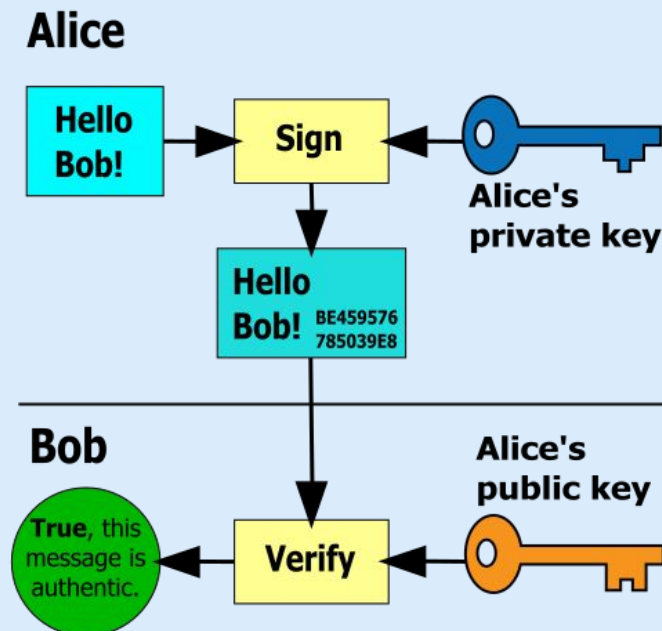
Asymmetric Cryptography



Use Case 2: Message authentication

- proves the authenticity and origin of a message.
- Recipient is sure of the origin of the message
- Sender can not deny having sent the message (**non-repudiation**)

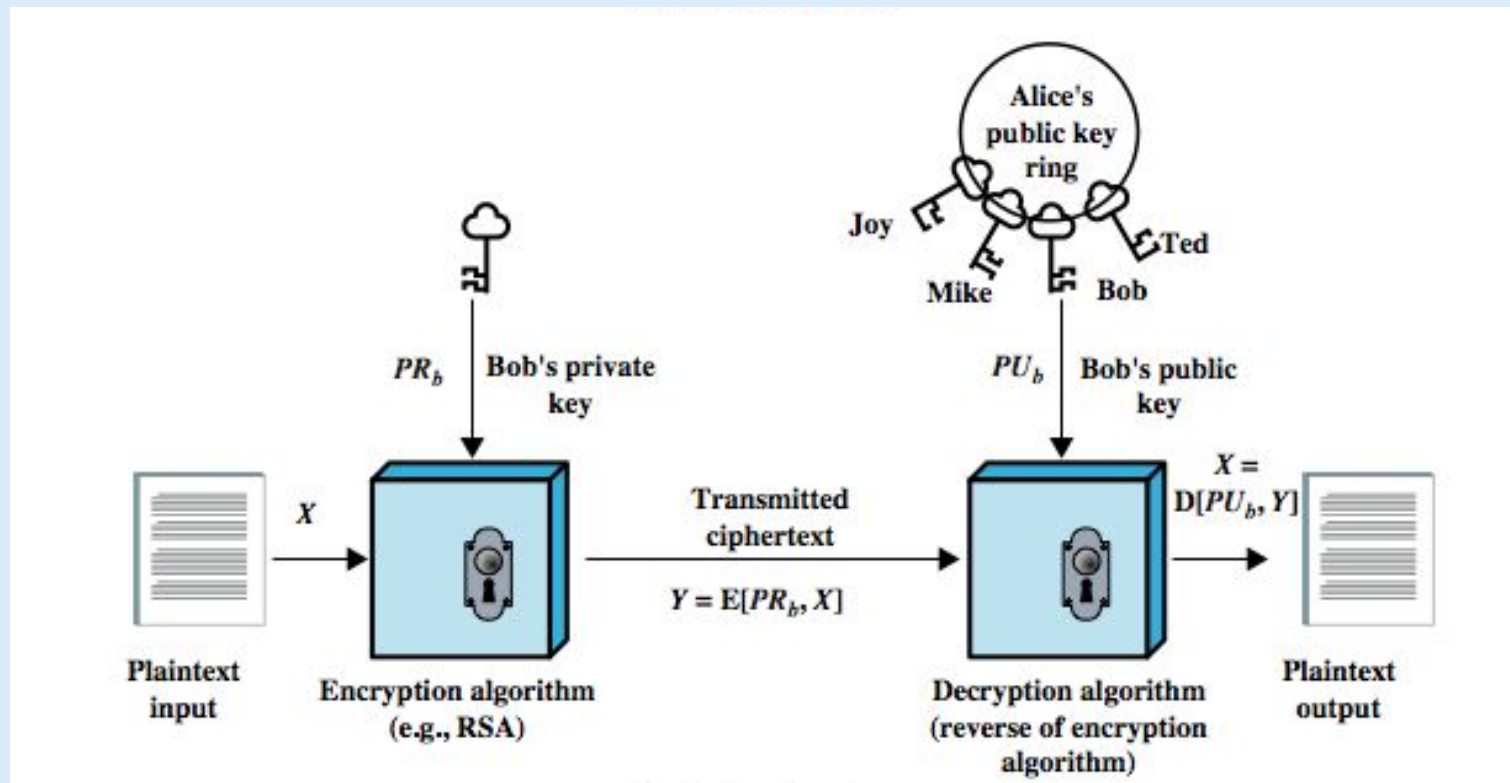
Message Authentication Code (MAC) produced using one's private key is called **Digital Signature**



Asymmetric Cryptography



Workflow of Bob sending a signed message. Alice (or anyone) can verify the message authenticity

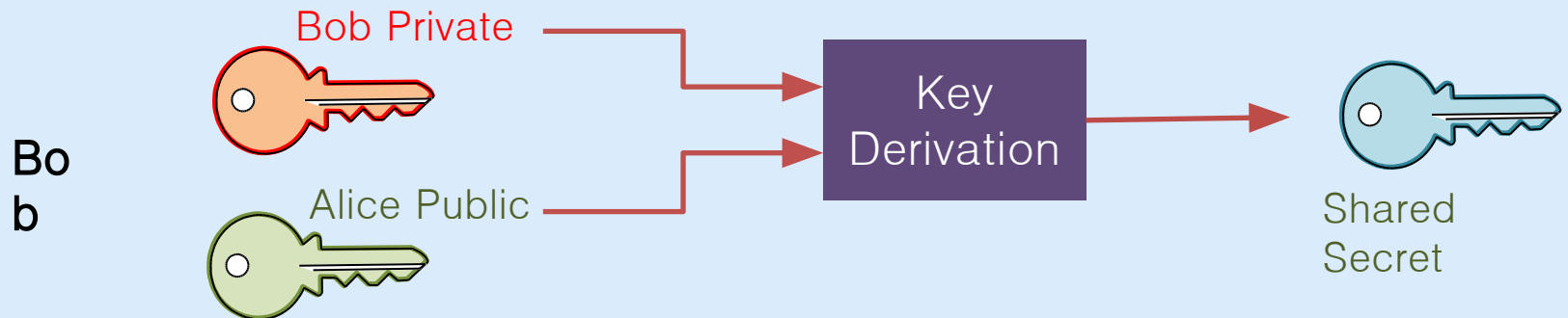
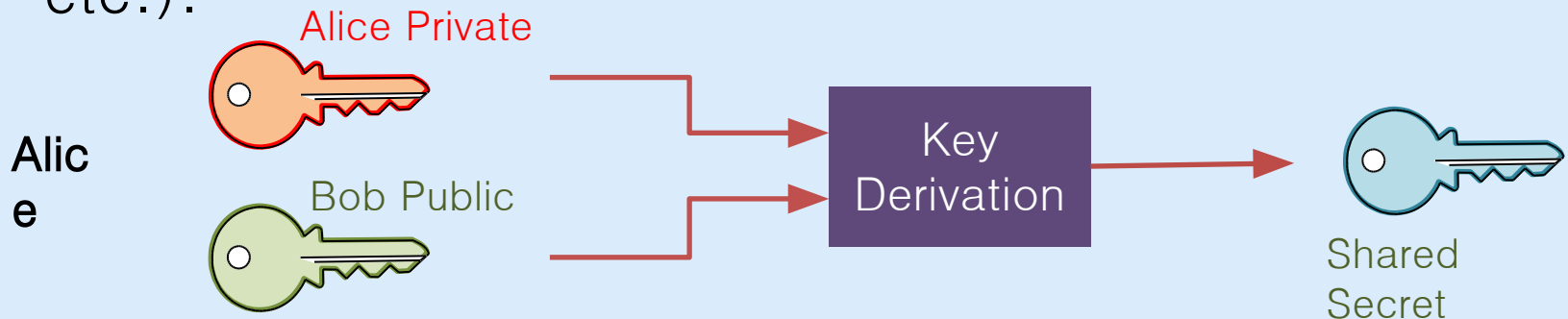


Asymmetric Cryptography



Use Case 3: Symmetric key exchange

- PKC can provide a secret channel for sharing a secret key K (to be used for symmetric crypto AES, DES etc.).



PKC Requirements



1. computationally easy to create key pairs
2. computationally easy for sender knowing public key to encrypt messages
3. computationally easy for receiver knowing private key to decrypt ciphertext
4. computationally infeasible for opponent to determine private key from public key
5. computationally infeasible for opponent to otherwise recover original message
6. useful if either key can be used for each role

PKC Algorithms



- RSA (Rivest, Shamir, Adleman)
 - developed in 1977
 - most widely accepted public-key encryption algo
 - need 2048+ bit keys
- Diffie–Hellman key exchange algorithm
 - only allows exchange of a secret key
- Elliptic curve cryptography (ECC) algorithms
 - Uses mathematical properties of elliptic curves
 - security like RSA, but with much smaller keys
- Post Quantum Cryptography (PQC) algorithms
 - latest family of algorithms that can withstand quantum computer attacks

RSA



- Invented by **R**on Rivest, Adi **S**hamir, and Len **A**dleman at MIT 1977
- Block size can be variable
- Key length can be variable
- Plaintext must be smaller than the key length
- Ciphertext block will be the length of the key
- Security depends on prime factorization problem
- Applications: secrecy and digital signatures

RSA: Background Math



Coprime numbers

- Two numbers are coprime (also called relatively prime) if the greatest common divisor (GCD) between them is only 1.
- A and B are coprime iff $\gcd(A, B) = 1$.
 - e.g. 6 and 11 are coprime because their gcd is 1 only
- It isn't necessary that the two numbers are independently prime — they just have to be prime to each other!

RSA: Background Math



Euler's Totient Function

- If n is a positive integer, φ (phi) function counts all the positive integers less than n that are co-prime to n .
- For example, $\varphi(10) = 4$
 - because 1, 3, 7, 9 are all coprime to 10
- For a **prime number** p , totient function is very straightforward: $\varphi(p) = p - 1$
- φ function is multiplicative over coprime numbers*
 - If n is a product of two prime numbers p and q ,
$$\varphi(n) = \varphi(pq) = \varphi(p) \varphi(q) = (p - 1)(q - 1)$$
 - e.g. $\varphi(77) = \varphi(7 \times 11) = 6 \times 10 = 66$

RSA Algorithm: Part 1



Key Construction

- Select two large primes: p and q , where $p \neq q$.
- Define $n = p \times q$
- Calculate totient of n , that is $\varphi(n) = (p - 1)(q - 1)$
- Select a number e that is coprime to φ and also less than φ , i.e.
 - $\gcd(\varphi, e) = 1$ and $1 < e < \varphi$
- Calculate d as the modular multiplicative inverse of $e \pmod{\varphi}$
 - $d \times e \bmod \varphi = 1$
- public key = (e, n) , private key = (d, n)

RSA Algorithm: Part 1



Key Construction Example

- Select two large primes: p and q , $p \neq q$. Let $p = 17$, $q = 11$
- Define $n = p \times q = 17 \times 11 = 187$
- Calculate $\varphi = (p - 1)(q - 1) = 16 \times 10 = 160$
- Select e ($1 < e < \varphi$), such that $\gcd(\varphi, e) = 1$; say $e = 7$
- Calculate d such that $d \times e \bmod \varphi = 1$
 - Use Euclid's algorithm to find $d = e^{-1} \bmod \varphi$
 - $160k + 1 = 161, 321, 481, 641, \dots$
 - check which of these is divisible by e
 - 161 is divisible by 7 giving $d = 161/7 = 23$
- Public key = $(7, 187)$
- Private key = $(23, 187)$

RSA Algorithm: Part 2



Encryption and Decryption

- To encrypt data, treat the plaintext M and ciphertext C as non-negative integers less than n .
- $\text{Key}_1 = (e, n)$ $\text{Key}_2 = (d, n)$

- To encrypt and decrypt

$$C = M^e \bmod n$$

$$M = C^d \bmod n$$

- Note that the roles of e & d are interchangeable.
 - i.e. $(x^d)^e \bmod n = (x^e)^d \bmod n$

RSA Algorithm: Part 2



Encryption and Decryption Example

- Messages to encrypt: $M_1 = 2$ and $M_2 = 5$
- Public key = $(7, 187)$
- Private key = $(23, 187)$

Encryption

$$C_1 = 2^7 \bmod 187 = 128 \bmod 187 = 128$$

$$C_2 = 5^7 \bmod 187 = 78125 \bmod 187 = 146$$

Decryption

$$M_1 = 128^{23} \bmod 187 = 2$$

$$M_2 = 146^{23} \bmod 187 = 5$$

Modular exponentiation with ordinary calculators



- Solving expressions of the form $128^{23} \bmod 187$ can involve working with extremely large numbers (like 128^{23}) which might go out of range of a typical calculator.
- In such cases, breakdown the exponent, and utilize the modular reduction property to make the calculation more manageable.

Property

$$(a \cdot b) \bmod m = \{(a \bmod m) \cdot (b \bmod m)\} \bmod m$$

Example

$$\begin{aligned} 128^{23} \bmod 187 &= (128^{10} \cdot 128^{10} \cdot 128^3) \bmod 187 \\ &= \{(128^{10} \bmod 187)(128^{10} \bmod 187)(128^3 \bmod 187)\} \bmod 187 \\ &= (166 \times 166 \times 134) \bmod 187 = 2 \end{aligned}$$

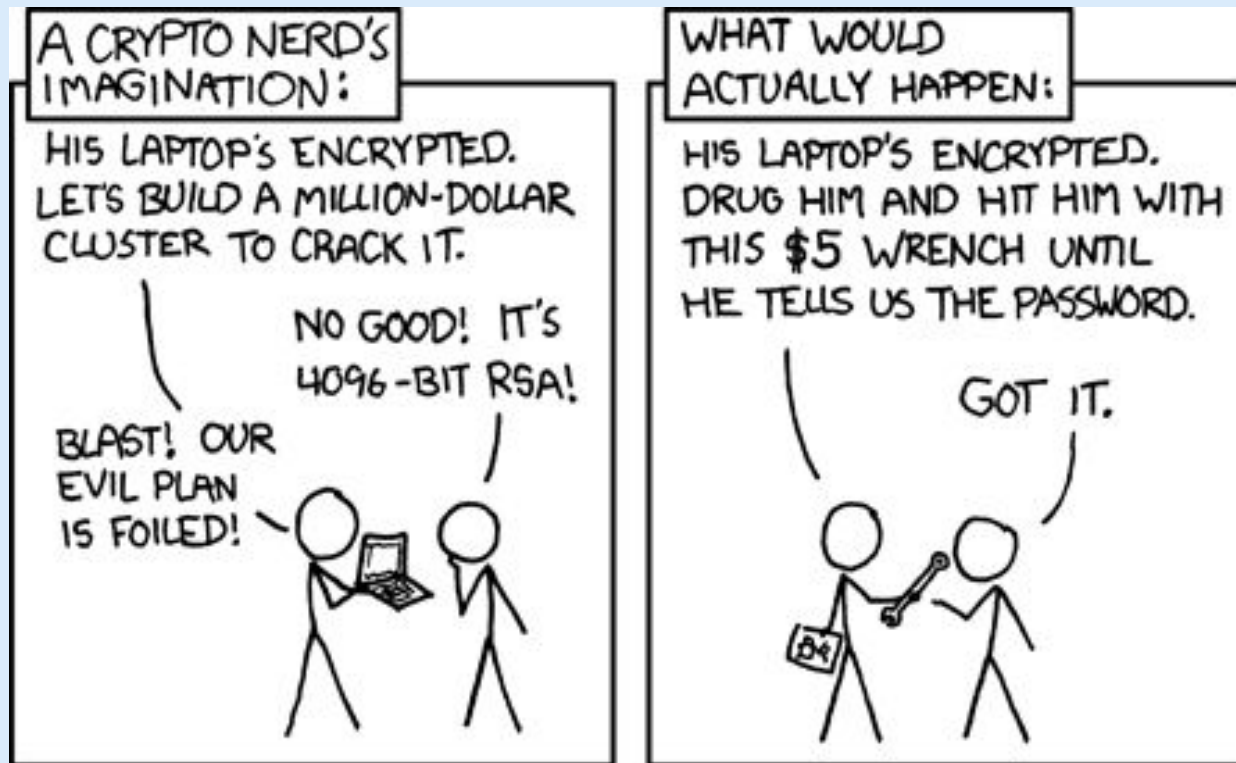
For larger exponents, refer to binary expansion and repeated squaring method.

RSA Encryption Strength



- Public and private keys are mathematically related, but that relationship is hidden from attackers.
- Both keys e and d are derived from p and q . Attacker only knows e and n . To figure out d , they must first find out φ , which requires knowledge of factors of n
 - since $\varphi = (p - 1)(q - 1)$ and $n = p \times q$
- Factorizing n seems trivial for small numbers, but when n is sufficiently large (i.e. several hundred digits!) **decomposing it into factors is computationally infeasible.**
- That's why RSA recommended key length (size of n) is, at least, 2048 bits, which is 617 decimal digits
 - individually p and q are 1024 bits each

RSA Encryption Strength



<https://xkcd.com/53>